Optimizing Fraud Detection in Financial Transactions using Machine Learning and Imbalance Mitigation

PROJECT REPORT[Phase-I/II/III]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

(Department of Computer Science and Engineering)

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY

BHOPAL (M.P.)

Submitted by

Aridaman Verma – 24U020055

Aman Verma – 24U020051

Under the supervision of

Dr. Saurabh Jain

Assistant Professor

CSE Department

November, 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Optimizing Fraud Detection in Financial Transactions using Machine Learning and Imbalance Mitigation"** has been satisfactorily completed by **Aridaman Verma (24U020055)** and **Aman Verma (24U020051)**. It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Technology during the academic year 2026-27.

Date: 2 November, 2026

Dr. Saurabh Jain
Assistant Professor
CSE Department
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Minor Project
Coordinator,
CSE Department,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



DECLARATION

We hereby declare that the following minor project report entitled "Optimizing Fraud Detection in Financial Transactions using Machine Learning and Imbalance Mitigation" is presented in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Saurabh Jain**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole for the award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Aridaman Verma (24U020055)

Aman Verma (24U020051)

ACKNOWLEDGEMENT

We would like to convey our gratitude towards everyone who has helped us to complete our minor project entitled “Optimizing Fraud Detection in Financial Transactions using Machine Learning and Imbalance Mitigation”. We would like to thank our Director Dr. Ashutosh Kumar Singh for providing all the resources required along with facilities to work in computer lab.

We express our gratitude towards our project guide Dr. Saurabh Jain for his practical tips, genuine suggestions regarding our code and experiments. He patiently guided us to help us improve this report.

We would like to thank our Minor Project Coordinator Dr. Rahul Chaurasia for organizing the evaluation timelines, clearing our doubts related to formatting and submission.

We would also like to show our gratitude to all the faculty members of the Department of Computer Science and Engineering for teaching us the basics of data structures and machine learning.

We would like to thank our friends for helping us brainstorm ideas while doing our experiments and identifying any bugs. We would also like to thank our families for being patient always and providing us with moral support during college.

We thank everyone who has helped us directly or indirectly.

Aridaman Verma (24U020055)
Aman Verma (24U020051)

AREA OF WORK

In today's world the majority of users will use a Tap, Swipe or UPI method. There is a lot of money being moved through these networks which means there is a high chance of fraud incurring, which causes costs.

Banks previously had rule based systems but scammers had very easy ways to bypass them. We created a whole pipeline in Python to see if we could use machine learning to detect fraud.

We started by cleaning the data. We did one hot encoding on the type of payments, and removed the customer ID's so that the model wouldn't learn the accounts.

We then used random under sampling to help balance out the large class imbalance. We trained 6 different models. Logistic Regression, Decision Tree, Gaussian Naive Bayes, Random Forest, XGBoost and KNN. We did 5 and 10 fold cross validation.

The best model was automatically saved and can be used to make predictions on new transactions. It can be run with a command line script or streamlit application.

The pipeline was based on the methodology of Al-dahasi et al [1]. Our goal was to test how well the pipeline works locally. This includes handling class imbalance. We also wanted to ensure there isn't any data leakage when doing so. Lastly we wanted to ensure our model finds fraud but doesn't cause too much of an issue for real users.

TABLE OF CONTENTS

Certificate	ii
Declaration	iii
Abstract	ix
1. Introduction	1
2. Literature Review	6
3. Problem Definition and Objectives	11
4. Proposed Algorithm	12
5. System Architecture and Workflow	14
6. Tools and Technology Used	17
7. Implementation and Coding	18
8. Result and Comparative Analysis	24
9. Conclusion and Future Scope	32
10. References	33
11. Plag report	34

LIST OF FIGURES

Fig 1: Layered architecture of the proposed fraud-detection system 14
Fig 2: End-to-end workflow (flowchart) of the training pipeline 16
Fig 3: Class distribution before and after random undersampling 19
Fig 4: Fraud rate by transaction type in the dataset 20
Fig 5: Correlation matrix of the transaction features 21
Fig 6: Model comparison under 5-fold cross-validation 25
Fig 7: Model comparison under 10-fold cross-validation 26
Fig 8: ROC-AUC of all models (5-fold and 10-fold) 26

LIST OF TABLES

Table 1: Summary of related work on ML-based fraud detection 6
Table 2: Features of the transaction dataset 18
Table 3: Hyperparameters used for each model 22
Table 4: Performance with 5-fold cross-validation and undersampling 24
Table 5: Performance with 10-fold cross-validation and undersampling 25
Table 6: Hold-out test confusion matrix for the best model 28
Table 7: Comparison with the source paper (10-fold validation) 28

ABSTRACT

In today's society most users will be using a Tap, Swipe or UPI payment method. With lots of money being passed through these networks there is a high chance of fraudulent activity which can incur costs.

Previously banks used rule based methods. But there were many easy ways for scammers to get around this. We implemented a full pipeline in Python to try and use machine learning to detect any fraud.

We first cleaned our data. We applied one-hot encoding to our payments types and also removed customer ID's so our model would not learn any accounts.

Then we used random under sampling to help with the high class imbalance. We trained 6 different models. Logistic Regression, Decision Tree, Gaussian Naive Bayes, Random Forest, XGBoost and KNN. We also performed 5 and 10 fold cross validation.

Our best model gets automatically saved and can be used to predict on new transactions. This can be used with a command line script or streamlit application.

Our pipeline was inspired by the methodology of Al-dahasi et al [1]. We wanted to demonstrate how effective our pipeline was locally. This included handling class imbalance. We also wanted to make sure there was no data leakage. Lastly we wanted to make sure our model detects fraud but does not cause too many issues for real users.

Keywords: Fraud detection, Machine Learning, Class imbalance, Random undersampling, Logistic Regression, XGBoost, SHAP, Payment Security

1. Introduction

1.1 Background and Need for Financial Fraud Detection

Digital payment systems have revolutionised how individuals, businesses and financial institutions move money. Customers can make quick and effortless transactions using card payments, mobile banking, QR code payments, e-wallets and instant payments. However, these easy methods can cause fraud to occur. Customer theft, identity theft, account takeover, unauthorised payments and colluding suspicious payments can occur. With an increase in electronic transactions it becomes difficult to manually review all the transactions. This means it is important to have automated fraud detection mechanisms in place to help reduce fraud within payment systems [1], [11], [12].

Fraud can cause businesses to lose a lot of money. Fraudulent payments can cause direct loss of money, costs to investigate, complaint from the customer, disruption to the service and lower confidence in digital payment systems. Some estimates say that businesses are losing around 4 trillion dollars a year due to fraud which equates to around 5% of their revenue. Additionally, there were over 790,000 cyber crime complaints made to the FBI Internet Crime Complaint Centre in 2020 and losses reached over 4.2 billion dollars. Financial crimes such as money laundering, corruption and tax evasion are estimated to cost around 2-5% of the global GDP [1].

Traditional fraud detection systems often use rule based methods. For example if a bank sees a payment above a certain amount, multiple attempts or strange combinations of payment details it will be flagged. These are easy to understand and easy to check. However, they are based around certain conditions. If fraudsters change the payment amount, split the payments into smaller payments, use different accounts or act like a normal customer then the payment may not be detected. Research into fraud detection in finance and risk management in banking has shown that it can be difficult to keep up with rule based detection as the number of transactions increases [1], [10], [11].

Machine learning offers another way to utilise the relationship between transactional features and historical fraud labels. Classification models can consider multiple factors from the transactional data (e.g. transaction amount, type, time step, etc). Previous reviews have looked into how classification, clustering, neural networks, Bayesian, and ensemble methods have been used in financial fraud detection and banking risk problems [11] [12]. In this case supervised learning methods are useful as the historical transactional data can

include a label to indicate if it was a fraudulent transaction or not. Supervised machine learning has been found to be useful within transactional networks in banking and other financial data sets [6] [13].

Multiple different algorithms can be used to represent relationships in the transactional data. Logistic Regression can be used as a basic probabilistic method. Decision Trees will split the feature space using decision rules. Gaussian Naive Bayes uses class probabilities and feature probabilities. K-Nearest Neighbours will compare new transaction to those nearest in the training set. Random Forest is an algorithm that combines multiple decision trees. XGBoost uses an ensemble of boosted trees [14] [18]. Previous studies have looked into using Naive Bayes, Support Vector methods, logistic models and ensemble tree methods in fraud detection [5] [10]. Deep Learning methods such as Long Short-Term Memory have also been investigated [7] [8]. This motivates looking into different models to compare their performance.

1.2 Why Fraud Detection Is a Challenging Machine-Learning Problem

A challenge within the problem is that there is often a large imbalance in transaction data between fraudulent and genuine payments. Within the Kaggle online payments data, there is approximately 0.108% fraudulent transactions, meaning if a classifier guesses that every row is genuine it will still achieve a high accuracy but will not detect any fraud [1] [2]. Any useful system should take into account how well the model detects the minority class, in this case fraudulent payments, and how many genuine payments have been falsely detected.

To overcome the imbalance problems there are multiple strategies. Over-sampling can be applied to increase the amount of samples in the minority class. This can be done using methods like Synthetic Minority Over-sampling Technique (SMOTE) where new samples of the minority class are generated [20]. Under-sampling decreases the number of majority class samples. Hybrid methods can also be used where both over and under-sampling are applied. Previous research has combined balancing techniques with Random Forest and CatBoost classifiers [3] [4]. Random Under-Sampling was applied in the paper to create a balanced dataset for training and 6 different machine learning models

were tested [1]. The imbalanced-learn package provides tools to perform this sampling [17].

The next issue is the misclassification costs. False negatives occur when fraudulent payments are predicted as genuine payments. This could mean an opportunity for financial loss. False positives occur when genuine payments are predicted as fraudulent. This could cause delays to authorised payments, extra checks with the customer and could decrease trust. Depending on what application the system has, it is important to consider the balance between false negatives and positives. Precision measures how many of the flagged payments are actually fraudulent. Recall measures how many fraudulent payments are detected. The F1-score considers both precision and recall. The ROC-AUC measures how well the model can classify between the two classes at different decision thresholds. All of these measures are used to evaluate the model compared to accuracy [1] [5] [6].

Fraudulent activity can vary over time. Fraudsters may change the amount of the transaction, type of payment or use new accounts. If the distribution changes then the model trained on historical data may become less effective. Therefor research in detecting financial fraud focuses on adaptive systems that can test the model on unseen data [1] [11] [12]. In real systems this could mean that the decision threshold and model may need to be reviewed if the transaction patterns change.

It is also important to understand why the model made a prediction. The fraud score could impact if payments are allowed, delayed or investigated. If a system only outputs a label it could be challenging to understand for an analyst to understand especially if the customer is genuine. SHAP (SHapley Additive exPlanations) provides values to indicate the contribution of each feature. This can help to understand what features impacted the model prediction [15]. SHAP was also used in the study to understand feature importance [1]. SHAP is used as part of this project to provide explainability of the chosen model.

1.3 Project Overview and Motivation

The present project develops a modular machine-learning pipeline for identifying potentially fraudulent financial transactions. Its methodology is based on the work of Al-dahasi et al. [1], which compares six classifiers and uses imbalance mitigation and feature engineering. The project implements the pipeline in Python and evaluates Logistic Regression, Decision Tree, Gaussian Naive Bayes, Random Forest, XGBoost, and K-Nearest Neighbours. The same

preprocessing and evaluation process is applied to each model so that their performance can be compared using a common set of metrics [1], [14], [16].

The project distinguishes between the full Kaggle online-payments file cited in the source study and the smaller dataset used for the experiments reported here. The full online-payments file is an available input option and is identified through its Kaggle source [2]. For a quick, self-contained demonstration, the project also includes a local generator that creates 6,000 synthetic rows following a PaySim-style transaction schema. The PaySim research describes the use of simulation to create synthetic mobile-money transactions when access to genuine financial records is restricted [19]. The locally generated sample is not the original Kaggle file and is not a complete run of the original PaySim simulator; it is a smaller development dataset created by the project code. This distinction is important when interpreting the results and comparing them with the source paper.

During preprocessing, the target variable is separated from the input features, account identifiers and selected columns are removed, and the transaction-type field is converted into binary features through one-hot encoding. Random Under-Sampling is then applied to reduce the majority class. The balanced data is divided into training and hold-out portions using stratified sampling, and standardisation is fitted on the training portion before being applied to the test portion. The use of a fixed preprocessing pipeline helps keep the feature representation consistent across the six classifiers [16], [17].

The models are evaluated using stratified five-fold and ten-fold cross-validation. Accuracy, precision, recall, F1-score, and ROC-AUC are recorded, and the best model is selected primarily using the mean F1-score, with recall and precision used as tie-breakers. The selected classifier is then evaluated on the hold-out test set. The training pipeline also creates plots, computes SHAP-based feature explanations when available, and saves the model together with the scaler and feature information. A command-line script supports batch prediction, while a Streamlit interface provides a simple way to enter a transaction and view its predicted class and fraud probability.

The objective is not to claim that a model trained on a small synthetic sample is ready for direct use in a bank. Instead, the project demonstrates an end-to-end workflow that can be executed locally, inspected, and tested without downloading the full dataset. Its results provide a controlled comparison of the selected algorithms on the generated sample. The report also compares those results with the source study and discusses why the scores differ, particularly

because the experiments use a smaller dataset with probabilistically generated fraud labels. A larger and more representative dataset would be required before drawing conclusions about real-world deployment.

The remainder of this report is organised as follows. Section 2 reviews related work on financial fraud detection, class imbalance, and explainability. Section 3 defines the problem and lists the project objectives. Section 4 presents the proposed algorithm and evaluation metrics. Section 5 describes the system architecture and workflow. Section 6 introduces the tools, software packages, machine-learning models, and dataset. Section 7 explains the implementation, while Section 8 presents the experimental results and compares them with the source paper. Section 9 concludes the report and lists possible extensions. Section 10 contains the references, followed by the plagiarism report.

2. Literature Review

From The Many Research Papers we have Observed The Following Conclusions**:**

2.1 Background

Now digital payment systems are embedded into everyday life, there are many ways that criminals can exploit them. Hackers stealing card details, phishing links and credentials have made it possible for users to be robbed. This causes huge costs for the banks to incur due to fraud which is then passed onto the customer. Research has therefore been focused on developing automatic fraud detection systems. [1] [11].

2.2 Rule-based systems

Rule engines utilise extensive IF-THEN rules that are manually created and updated by staff members within a bank.Whilst it can be useful for detecting simplistic attacks it is unable to detect more complex feature combinations. If a user sends the majority of their balance to a new account straight after logging in then this may not be picked up by a simplistic rule. Machine Learning allows all features to be mapped to a high dimensional space. This allows for flexible decision boundaries to be created and fraudulent transactions to be detected. [1],[10],[11].

2.3 Previous works

A summary of some of the key studies into using Machine Learning techniques in Financial Fraud is provided below in table 1. This includes banking risk assessments, supervised transaction modelling and alternative techniques for detecting fraud [3]-[13].

Table 1: Summary of related work on ML-based fraud detection

Study	Technique	Key point reported
Al-dahasi et al. (2025) [1]	Six ML models + undersampling + SHAP	Kaggle online-payments data; XGBoost and Random Forest best (F1 up to 0.99)

Study	Technique	Key point reported
Alfaiz and Fati (2022) [3]	AllKNN undersampling + CatBoost	Two-stage selection of best learners on real credit-card data; AUC 97.94%, recall 95.91%, F1 87.40%
Taneja et al. (2019) [4]	SVM-SMOTE balancing + Random Forest	Best of several balancing/ensemble combinations: recall 0.80, precision 0.91, F-score 0.85
Gupta et al. (2021) [5]	Naive Bayes, SVC, LR, RF with random sampling	284,807 transactions; ~99.9% accuracy but precision/recall of only 56–93% and 63–80%
Khosravi et al. (2023) [6]	Supervised ML on banking transaction network	46,316 transactions, 25 features; features with Pearson correlation above 0.8 removed
Alghofaili et al. (2020) [7]	LSTM deep learning	Credit-card data; 99.95% accuracy reported in under a minute
Xiuguo and Shengyong (2022) [8]	LSTM and GRU with text + numeric features	Financial-statement fraud in 5,130 Chinese firms; 94.98% (LSTM) and 94.62% (GRU)
Ashfaq et al. (2022) [9]	XGBoost and Random Forest on blockchain	Fraud detection for Bitcoin-style transactions using accuracy, precision and AUC
Zareapoor and Shamsolmoali (2015) [10]	Bagging ensemble classifier	Credit-card fraud detection; ensemble bagging compared against single classifiers (SVM, KNN, Naive Bayes, decision tree)
Leo et al. (2019) [11]	Literature review of ML in banking	Reviews ML applications across banking risk management, including credit, market, operational and fraud-related risk
Mashrur et al. (2020) [12]	Survey of ML for financial risk management	Surveys ML techniques and open challenges in financial risk, including fraud detection
Kalbande et al. (2021) [13]	ML-based fraud detection system	Proposes a supervised ML pipeline for detecting fraudulent transactions

2.4 Class Imbalance

The raw transaction data was highly class imbalanced. Training a classifier without considering this bias will result in it learning to classify to the most common class and therefore missing many fraudulent transactions. Therefore

we must consider metrics such as the precision, recall and F1 score for the fraud class [1], [5].

There are various ways the data can be made balanced. Oversampling can be used where the minority class is increased. This can be done using SMOTE to generate synthetic samples from observations in the minority class [20]. Undersampling can also be used to reduce the majority class. Hybrid methods can be used to combine both methods. Various studies have been done into balancing methods with ensemble classifiers [4]. All-Nearest K-Nearest Sampling has been combined with CatBoost [3]. Random Under-Sampling has been used to produce a balanced dataset to compare different classifiers [1]. This method was used in this project to create a balanced dataset. This was done using the RandomUnderSampler implementation provided by imbalanced-learn [17].

2.5 Explainability

The prediction from a fraud detection model should be explainable enough so that if a particular account/payments are flagged for review a reviewer is able to know why they were flagged. SHAP gives a consistent way to explain how much each feature contributes to a particular prediction. It can be used to explain which features are most important in a dataset [15]. Al-dahasi et al [1] used SHAP to better understand which transaction features were contributing to their fraud detection experiments. As part of the explainability process, SHAP has been used within this project (if it can be successfully computed within the local environment).

2.6 Source Paper

The primary source for this project is the study on detecting fraud using machine learning and mitigation of imbalances [1]. The study uses a dataset about online payments from Kaggle [2] and performs various types of feature engineering and Random Under-Sampling. From here they compare 6 different classifiers for machine learning. Two of the most powerful models were found to be XGBoost and Random Forest. However, the Gaussian Naive Bayes performed much worse with a significantly worse F1-Score. The models were evaluated using precision, recall, F1-Score and ROC-AUC. SHAP was also applied to look into how well the models can explain themselves [1].

This project uses the general method and comparison of models as outlined in the source paper. However, the experiments have been completed using a smaller dataset that has been created. This means that the Kaggle dataset [2]

can still be used as an option; however, the built in sample will follow a similar structure to PaySim [2][19]. As a result, the scores mentioned in the source paper will be used as a comparison only. Therefore, differences in the sample size, labelling and experimentation should be noted.

2.7 Research Gap

- Lack of an easily reproducible, modular implementation for resource-constrained systems.
- Difficulty running large transaction datasets on typical student computers.
- Need for an automated cross-validation and model-selection workflow.
- Need for a lightweight offline dataset generator for testing.

3. Problem Definition And Objective

Problem Statement

Modern digital banks deal with huge volumes of payments but only a small proportion of those are fraudulent. Static rule based filters can easily be adapted by thieves. This causes machine learning classifiers to fail due to the imbalance of the data. All payments are predicted to be genuine. We want to produce a reliable classifier that can identify fraudulent payments without causing inconvenience to real users. We want high recall and reasonable precision. The system should be modular, fast to train and easily interpretable.

Objectives

- This project aims to create a machine learning pipeline to detect fraudulent activity. This is achieved by performing some data pre-processing, balancing the classes using random under-sampling and comparing 6 different classifiers using cross-validation and performance metrics.
- The best model can be automatically selected and then assessed using a final test set. The model can then be saved using Joblib. There is also a command line tool to perform batch predictions and a Streamlit dashboard. It also contains code to generate synthetic data to allow independent testing.

4. Proposed Algorithm

Algorithm: Imbalanced Data Aware Machine Learning Pipeline to Detect Fraud

Input:

The transaction table consists of a series of columns describing the transactions. The columns are: step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud and isFlaggedFraud. The six models M are fed into the algorithm and are tested using K folds (either 5 or 10) to see how well they learn.

Output:

Multiple outputs are created such as a set of models, a scaler that has been trained, cross validation metrics, a confusion matrix and prediction scripts. Everything is saved so that the model can be tested on new inputs at any time.

Phase One: Preprocessing

1. The script will attempt to find the data/raw/online_payments.csv file on the Hard Drive. If this file cannot be found then it will generate 6000 rows of data similar to that of PaySim.
2. The target variable is separated. This is done by using the 'isFraud' column as our target variable (y). All other columns are used as the feature matrix (X).
3. Irrelevant columns are removed. The columns 'nameOrig', 'nameDest', 'isFlaggedFraud', 'newbalanceOrig' and 'newbalanceDest' are dropped. This stops the model from learning the names of the accounts and avoids label leakage. It keeps the model fair and prevents it from cheating on new rows of data. It will ensure that the model learns true patterns.
4. Transaction types are hot encoded. This generates 5 new binary variables (CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER) from the type column. This is done to convert the text into numbers which are easier to understand by the model.

Phase Two: Addressing Imbalance

1. Under sampling is used on the non fraud class. The under sampler will reduce the amount of examples in the majority class to result in a 1:1 ratio in the dataset (fraud and non fraud). This can be done using the RandomUnderSampler utility from imbalanced-learn [17].
2. The dataset is now balanced so it can be split into training data (80%) and test data (20%). Using stratified sampling with a set random seed.
3. The StandardScaler will be fitted to the training split. This will scale both the training and test splits. Ensuring that future test data is not shown.

Phase Three: Evaluate Models

1. All 6 algorithms will be trained using stratified 5 and 10 fold cross validation. The models will be evaluated using metrics like mean Accuracy, Precision, Recall, F1 Score and ROC-AUC to find out which model performs the best for detecting fraudulent transactions. The models will be tested carefully on each fold.

2. Models will be ordered by their mean F1 score, from cross validation. If there are multiple models with the same F1 score then these will be ordered by Recall and Precision.

Phase Four: Final Test + Explain

1. The optimal model will be retrained using the entire 80% of the training data and will be evaluated using the untouched 20% of testing data.

2. ROC curves, confusion matrices and SHAP values will be produced to show what features are driving the fraud score. SHAP provides a method of attributing importance to the different features [15].

Phase Five: Deploy

1. The optimal model, scaler and features are saved to a joblib file.

2. During inference payments are matched up, scaled and then scored. If they have a predicted fraud probability of 0.5 or higher they will be marked as fraudulent. If they have a predicted fraud probability less than 0.5 they will be allowed through as safe.

End of Algorithm

Metrics Used for Evaluation

- The following standard metrics will be used. Fraud will be taken as the positive class. They are commonly used in comparative fraud detection studies [1] [5] [6].

- Accuracy – This tells us the proportion of correct predictions out of all data (TP+TN)/Total. Not reliable when there is an imbalance in the data.

- Precision – The proportion of fraudulent payments that have been identified (TP)/(TP+FP).

- Recall – The proportion of fraudulent payments that have been identified (TP)/(TP+FN).

- F1 Score – Calculates the harmonic mean between precision and recall (2 x Precision x Recall) / (Precision + Recall). Penalises imbalanced models.

- ROC-AUC – This tells us how well the model is able to distinguish between the two classes. It does this by calculating the area under the true positive rate versus false positive rate curve at all thresholds.

5. System Architecture And Workflow

We have divided the system into 5 distinct layers. This is the same as how our package has been structured in Python. This makes it very straightforward to test our system. For example, we can swap out our synthetic dataset for the full Kaggle file without modifying any of our training/inference code.

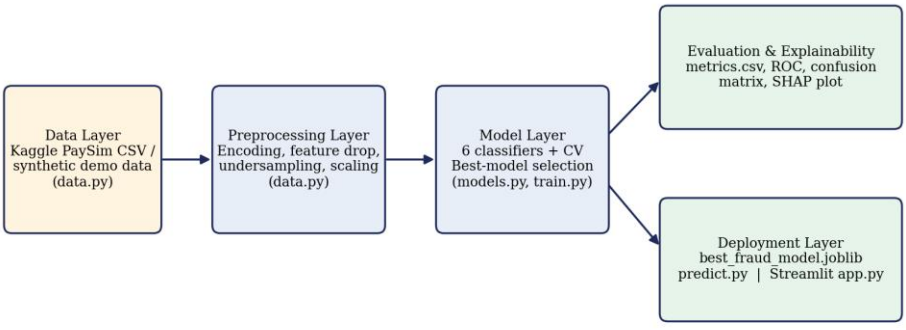


Figure 1: Layered architecture of the proposed fraud-detection system

Figure 1 presents the system as five connected layers: data input, preprocessing, model training, evaluation and explainability, and deployment. The separation of these layers makes it possible to change the input dataset or model without redesigning the complete application.

1. Data layer

The data layer loads the payment data. If the data/raw/online_payments.csv file exists, it will be loaded straight away. If not, a synthetic dataset of 6000 rows will be created using the create_demo_dataset() function. This will generate rows in the exact format that PaySim uses. By doing this we can test our code as we can create our own data without having access to the web. The full Kaggle file is cited below [2] [19].

2. Preprocessing layer

The preprocessing stage cleans our data. We remove the string ID and collinear balance data. The payment type data is one hot encoded which gives us 5 binary variables. We also balance our classes by using

RandomUnderSampler and fit the StandardScaler of our training data. This ensures we have a clean dataset.

3. Model layer

The model layer of the code contains the 6 classifiers. The hyperparameters were set as described by Al-dahasi et al. The train.py module conducts 5 and 10-fold cross validation to identify the best performing model, based on the highest F1 score.

4. Evaluation & explainability layer

The evaluation stage records performance metrics for the model. The validation results are recorded into the report/metrics.csv file. We do some ROC curves, hold out confusion matrices and SHAP functions to visualise the important features and understand why each call was made.

5. Deployment layer

On the deployment layer, we wrap up our model, feature column names and scaler into a model file which will be saved into models/best_fraud_model.joblib. We can then use predict.py to score batch files of CSVs or use app.py to test individual payments using a web dashboard. This makes it very quick, easy to use and clean. An overview of the workflow can be seen in figure 2.

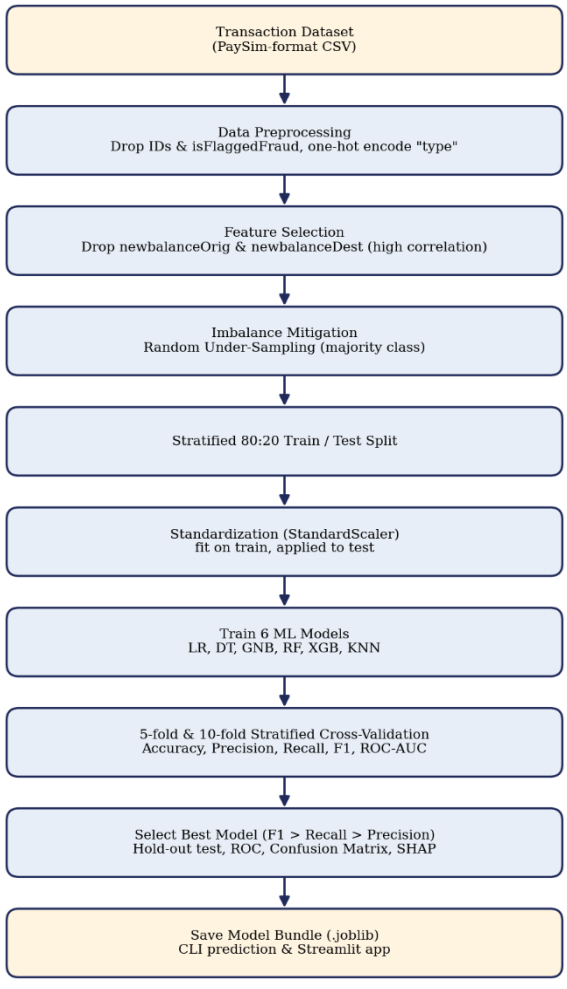


Figure 2: End-to-end workflow (flowchart) of the training pipeline

Figure 2 shows the order of operations in the training pipeline. Transaction data is preprocessed, redundant features are removed, the class distribution is balanced, and the data is split and standardised before six classifiers are evaluated using stratified cross-validation. The selected model is then tested, explained, saved, and made available through the prediction interfaces.

6. Tools And Technology Used

6.1 Programming language and environment

This project was built using Python 3.10+ due to the wide variety of Machine Learning and Data Science packages available. All packages have been installed in a virtual environment. The package can be installed in editable mode using the pyproject.toml file with the command pip install -e.

6.2 Machine-learning models used

- Logistic Regression is a linear classifier which makes use of the logistic function to predict the probability of classes. In this project the model uses L1 regularisation and the solver C = 10. The model is a simple baseline to compare against [16].
- Decision Tree is a model which splits the features space by asking a number of questions about the features. Entropy was used to split the data [16].
- The Gaussian Naive Bayes model uses Bayes theorem. The model has the assumption that each feature follows a Gaussian distribution for each class. The variance smoothing parameter was set to 1.2328 x 10-7. This model provides a quick baseline [16].
- Random Forest is an ensemble model that averages the results from several Decision Trees. Within this project, 50 trees were used so that the variance can be reduced [18].
- XGBoost is a sequential ensemble of Decision Trees. The model attempts to correct the mistakes made by previous trees. It is a gradient boosting technique which can scale [14].
- K-Nearest Neighbours is a model which uses the labels from nearby training examples to label a transaction. Nine neighbours were used and weighted by distance [16].

This project includes a copy of the Kaggle Online Payments Fraud Detection file. This is the complete dataset mentioned in the source study and contains 1,042,574 entries. It has a fraud rate of ~0.108% [1] [2]. However, the experiments described within this document have been completed using the projects built in synthetic sample of 6,000 entries. This local generator follows the PaySim format for transactions [19]. If you have a data/raw/online_payments.csv file you can use this as an input for the pipeline. The results in section 8 should be interpreted as being based on the synthetic data unless you have used the full data file.

7. Implemention And Coding

7.1 Project structure

```
implementation-of-ml-minor-project/
|-- app.py                    # Streamlit web interface
|-- requirements.txt, pyproject.toml
|-- data/sample/demo_transactions.csv   # synthetic dataset
|-- reports/metrics.csv         # cross-validation + hold-out scores
|-- docs/                     # implementation notes, source-paper note
`-- src/fraud_detection/
   |-- config.py               # paths, constants, columns to drop
   |-- data.py                 # dataset generation, preprocessing, sampling
   |-- models.py               # six classifiers with tuned hyperparameters
   |-- train.py                # cross-validation, selection, plots, saving
   `-- predict.py              # batch prediction from a CSV file
```

7.2 Dataset used in the experiments

All experiments within this report used the 6000-row synthetic dataset provided by create_demo_dataset(). The data has been created using the PaySim style schema [19], with 374 (6.23%) fraudulent transactions. Transactions have been randomly generated using the specified probabilities for each transaction type: 22% CASH_IN, 34% CASH_OUT, 4% DEBIT, 30% PAYMENT, and 10% TRANSFER. Values for transaction amount and balances have been log-normally distributed. Fraud labels have also been randomly assigned to transactions. A higher proportion of fraud has been generated within TRANSFER and CASH_OUT transactions. This dataset has been created locally to be used for testing the pipeline and should not be confused with the larger Kaggle online payment dataset [1][2].

Table 2: Features of the transaction dataset

Feature	Description
step	Time unit; one step is one hour (744 steps = 30 days)
type	Transaction type: CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER
amount	Amount of the transaction in local currency
nameOrig / nameDest	Identifiers of the sender and of the recipient (dropped)
oldbalanceOrg	Sender balance before the transaction
newbalanceOrig	Sender balance after the transaction (dropped: high correlation)
oldbalanceDest	Recipient balance before the transaction

Feature	Description
newbalanceDest	Recipient balance after the transaction (dropped: high correlation)
isFraud	Target label: 1 = fraudulent, 0 = genuine
isFlaggedFraud	Rule-based flag for transfers above 200,000 (dropped)

Table 2 describes the transaction dataset features, including transaction time, type, amount, and sender/recipient balances. It identifies isFraud as the target variable and highlights the removal of identifiers, highly correlated balance variables, and rule-based flags during preprocessing.

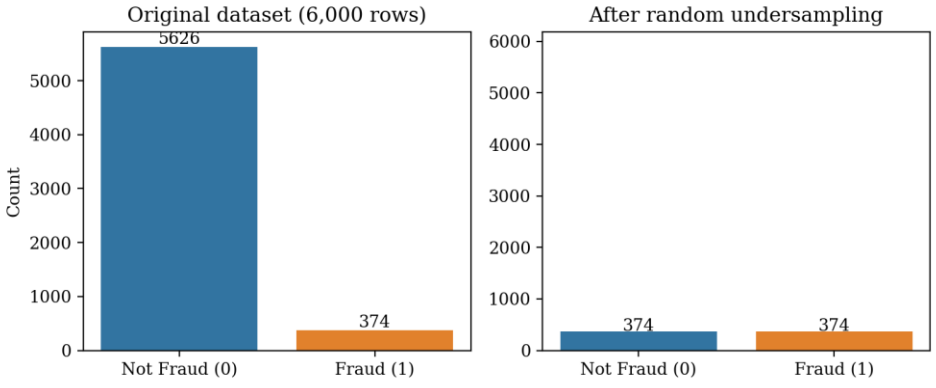


Figure 3: Class distribution before and after random undersampling

Figure 3 compares the class counts in the generated dataset before and after Random Under-Sampling. Before sampling, the dataset contains 5,626 legitimate transactions and 374 fraudulent transactions. After sampling, both classes contain 374 rows, producing a balanced dataset of 748 transactions for the subsequent experiment [17], [19].

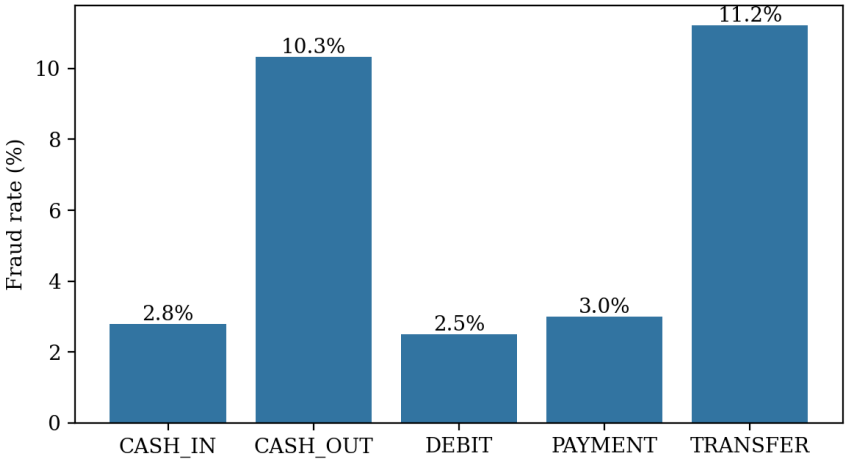


Figure 4: Fraud rate by transaction type in the dataset

Figure 4 compares the fraud rate across transaction types in the locally generated sample. TRANSFER has the highest displayed rate (11.2% of 579 rows), followed by CASH_OUT (10.3% of 2,052 rows). PAYMENT, CASH_IN, and DEBIT have lower displayed rates of approximately 3.0%, 2.8%, and 2.5%, respectively. These values describe the generated sample only and should not be interpreted as universal fraud rates for real payment systems [19].

7.3 Data preprocessing

The prepare_features() function performs some preprocessing on the data. It first extracts the target label y and then drops all the columns given in DROP_COLUMNS. We also encode the type text using one-hot encoding. For each categorical variable not present in the data we add an extra column consisting of all zeros. This ensures that we have the same amount of features in both our training and testing data. The pipeline uses pandas and scikit-learn compatible numerical inputs [16].

```
DROP_COLUMNS = ["nameOrig", "nameDest", "isFlaggedFraud",
                "newbalanceDest", "newbalanceOrig"]


def prepare_features(df):
    y = df["isFraud"].astype(int)
    x = df.drop(columns=["isFraud"])
    x = x.drop(columns=[c for c in DROP_COLUMNS if c in x.columns])
```

```
    x = pd.get_dummies(x, columns=["type"], prefix="type", dtype=int)
    for t in TRANSACTION_TYPES:              # guarantee all 5 columns
        if f"type_{t}" not in x.columns: x[f"type_{t}"] = 0
    return x.select_dtypes(include=["number","bool"]).astype(float), y
```

To identify redundant balance fields, Pearson correlations were calculated across the numeric transaction features. The resulting matrix is shown in Figure 5. The analysis was used to guide the removal of the post-transaction balance columns from the model inputs.

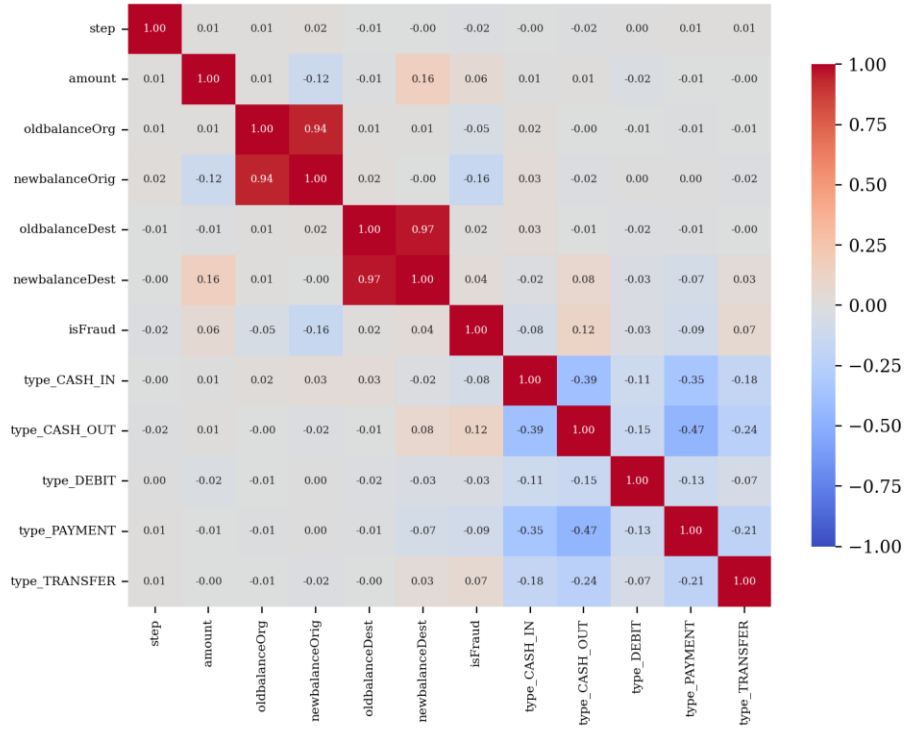


Figure 5: Correlation matrix of the transaction features

Figure 5 is a Pearson-correlation heatmap for the transaction features. It highlights very strong positive relationships between the pre-transaction and post-transaction balance fields, with some correlations in the range of 0.98 to 1.00. These highly related balance variables motivated the removal of selected post-transaction columns to reduce redundancy and the risk of label leakage.

7.4 Imbalance mitigation, splitting and scaling

The function build_train_test_data() uses the RandomUnderSampler class to balance the data by decreasing the transactions in the majority class [17]. This creates a balanced dataset with 374 instances for each class, resulting in 748 rows. The data is then split into an 80/20 train-test split using stratify. This

creates a training set with 598 samples and a hold-out set with 150 samples (75 fraudulent and 75 legitimate). The StandardScaler is fitted to the training portion only and then used to scale the test portion.

```
sampler = RandomUnderSampler(random_state=42)
x_bal, y_bal = sampler.fit_resample(x, y)
x_train, x_test, y_train, y_test = train_test_split(
    x_bal, y_bal, test_size=0.20, stratify=y_bal, random_state=42)
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)   # fit on train only
x_test_scaled  = scaler.transform(x_test)
```

7.5 Models and hyperparameters

The hyperparameters used in models.py are based on those reported in the paper by Al-dahasi et al [1]. This means that it is possible to concentrate on replication of the evaluation process rather than having to perform a computationally intensive grid search for optimal hyperparameters on our local sample.

Table 3: Hyperparameters used for each model

Model	Hyperparameters
Logistic Regression	C = 10, L1 regularization (liblinear solver), max_iter = 1000
Decision Tree	criterion = entropy, max_depth = None, min_samples_leaf = 1, min_samples_split = 5
Gaussian Naive Bayes	var_smoothing = 1.2328×10^{-7}
Random Forest	bootstrap = True, max_depth = 20, min_samples_leaf = 1, min_samples_split = 2, n_estimators = 50
XGBoost	learning_rate = 0.1, max_depth = 5, n_estimators = 200, objective = binary:logistic
K-Nearest Neighbours	n_neighbors = 9, weights = distance, algorithm = auto

Table 3 summarizes the hyperparameters used for the six machine learning classifiers to ensure consistent model training and comparison. These settings control model complexity, learning behavior, and prediction performance.

7.6 Cross-validation and model selection

For each classifier we used stratified K-fold cross validation. We divided our data into K=5 and K=10 folds which we calculated mean scores for. The pipeline then chooses the model with the highest F1 score, if there are tied scores it will use recall and precision to select the best model. The best

performing model is then trained on all our training data (598 rows) and then tested on our test data (150 rows).

```
cv = StratifiedKFold(n_splits=folds, shuffle=True, random_state=42)
for name, model in get_models().items():
    scores = cross_validate(model, x_train, y_train, cv=cv, n_jobs=-1,
        scoring={"accuracy":"accuracy","precision":"precision",
                 "recall":"recall","f1":"f1","roc_auc":"roc_auc"})
    rows.append({"model": name, "folds": folds,
                 "f1": scores["test_f1"].mean(), ...})


best_row = metrics.sort_values(["f1","recall","precision"],
                               ascending=False).iloc[0]
```

7.7 Explainability

The try_plot_shap() function will calculate SHAP values [15] using up to 300 training examples from the best model. This will also create a summary plot of the most impactful features. We have added try/except to ensure the model will still be trained even if there are any issues with calculating SHAP. You can also use --skip-shap in order to speed up your tests.

7.8 Prediction and web application

This script loads our saved joblib file. We clean the features of our input CSV file and make predictions with our model. We save our predicted labels and fraud probability. Our app.py uses streamlit to create a web application. This allows users to input information about their transactions and instantly get predictions on whether their transaction is fraudulent.

```
python -m fraud_detection.train                    # train & evaluate
python -m fraud_detection.predict --input data/sample/demo_transactions.csv \
                                  --output reports/predictions.csv

streamlit run app.py                               # web interface
```

8. Result And Comparative Analysis

Here are our results from our experiments on the synthetic data. The dataset has been balanced such that there are 598 training samples and 150 hold out samples. The 5-fold and 10-fold cross validation scores for all of our 6 classifiers can be seen below:

8.1 Cross-validation results

The cross-validation has been performed using both 5 folds and 10 folds to ensure consistency in the results. Random Under-Sampling was used to obtain balanced classes. All six models have been compared using various metrics (Accuracy, Precision, Recall, F1-Score and ROC-AUC) to determine which gives the best results for detecting fraudulent transactions.

Table 4: Performance with 5-fold cross-validation and undersampling

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6688	0.6579	0.7092	0.6811	0.7051
Decision Tree	0.5552	0.5708	0.4986	0.5286	0.5525
Gaussian Naive Bayes	0.6555	0.6432	0.7025	0.6700	0.6679
Random Forest	0.6172	0.6133	0.6489	0.6293	0.6769
XGBoost	0.5869	0.5851	0.5921	0.5878	0.6532
K-Nearest Neighbours	0.6354	0.6265	0.6855	0.6535	0.6783

Table 4 compares the six classifiers using 5-fold cross-validation with undersampling. Logistic Regression performs best overall, achieving the

highest accuracy (0.6688), F1-score (0.6811), and ROC-AUC (0.7051), while Decision Tree shows the weakest performance.

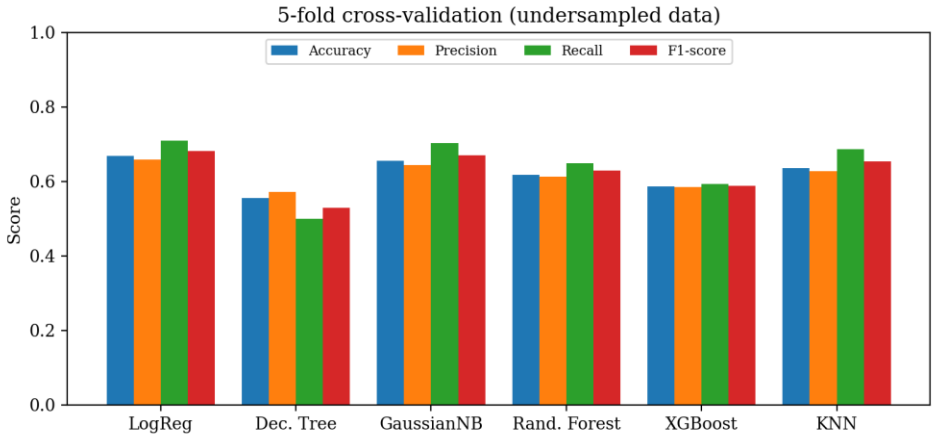


Figure 6: Model comparison under 5-fold cross-validation

Figure 6 compares accuracy, precision, recall, and F1-score for the six classifiers under stratified five-fold cross-validation. Logistic Regression has the highest F1-score (0.6811) among the tested models, followed closely by Gaussian Naive Bayes (0.6700). Decision Tree has the lowest F1-score (0.5286) in this experiment.

Table 5: Performance with 10-fold cross-validation and undersampling

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.6638	0.6513	0.7090	0.6771	0.7078
Decision Tree	0.5703	0.5818	0.5218	0.5476	0.5759
Gaussian Naive Bayes	0.6538	0.6416	0.7022	0.6691	0.6562
Random Forest	0.6087	0.6001	0.6523	0.6243	0.6710
XGBoost	0.6170	0.6131	0.6355	0.6218	0.6636
K-Nearest Neighbours	0.6388	0.6257	0.7023	0.6602	0.6725

Table 5 presents the performance of six classifiers using 10-fold cross-validation with undersampling. Logistic Regression achieves the best overall

results, with the highest accuracy (0.6638), F1-score (0.6771), and ROC-AUC (0.7078), showing consistent performance compared with the 5-fold results.

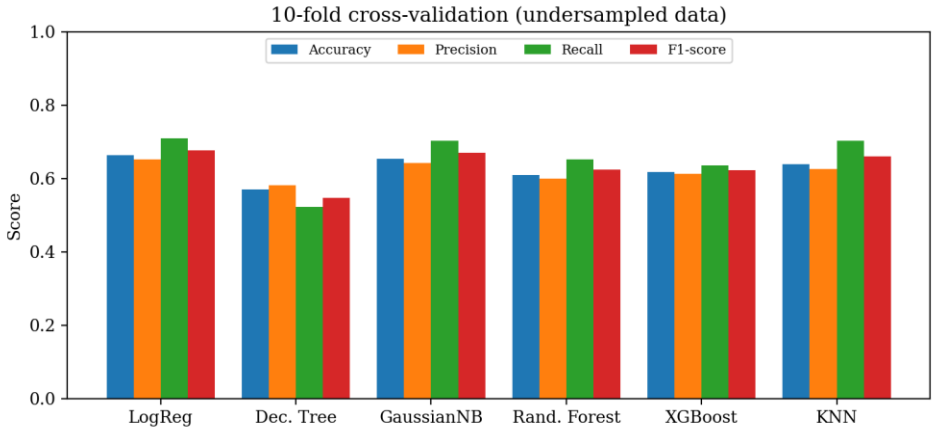


Figure 7: Model comparison under 10-fold cross-validation

Figure 7 presents the same four evaluation metrics under stratified ten-fold cross-validation. Logistic Regression again achieves the highest F1-score (0.6771), while Gaussian Naive Bayes and K-Nearest Neighbours also show comparatively strong results. The ranking is broadly consistent with the five-fold experiment.

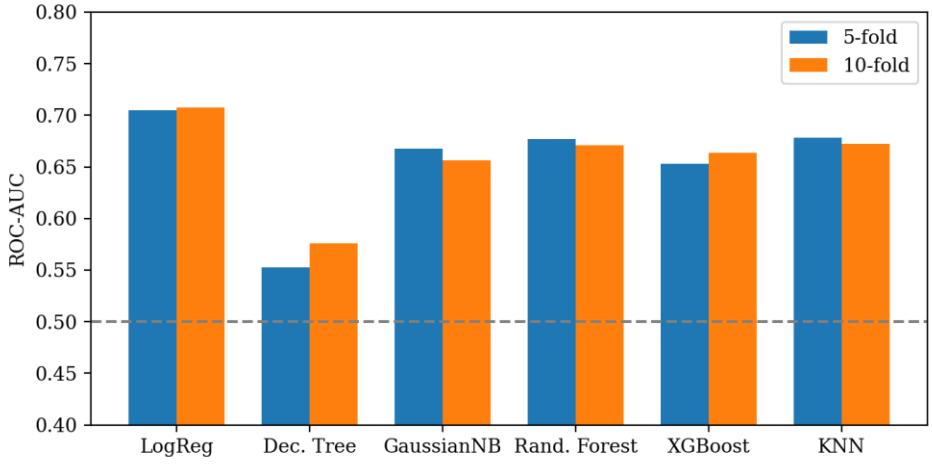


Figure 8: ROC-AUC of all models (5-fold and 10-fold)

Figure 8 compares ROC-AUC values from the five-fold and ten-fold experiments. Logistic Regression records the highest values in both settings (0.7051 and 0.7078), indicating the strongest class-separation performance among the six models on this generated sample. The close paired bars also show that the ROC-AUC results are similar across the two cross-validation settings.

Key observations from conducting cross validation:

- Logistic Regression was the most consistent model with the highest F1 score across both the 5 fold (0.681) and 10 fold (0.677) cross validations. In terms of ROC-AUC it also had the highest score (0.705 and 0.708). Because of its consistency it was deemed the best model.
- The tree based models performed poorly. Both Decision Trees, Random Forests and XGBoost had lower F1 scores between 0.528 and 0.629. This is because there is only a small amount of training data (598 rows) and label noise. With only a small amount of data tree models will often suffer from overfitting. A linear decision boundary with L1 regularisation works much better with limited data.
- Consistent results across different number of folds. There wasn't a large difference between the 5 fold and 10 fold results. The Logistic Regression only had a slight difference between 0.681 and 0.677. The Random Forest only had a slight difference between 0.629 and 0.624. This shows that our results are consistent.
- The hold out test set results were consistent. The Logistic Regression had an F1 score of 0.679 on 150 unseen test samples. This is very consistent with the 10 fold cross validation result of 0.677. This shows that the model did not suffer from overfitting and can generalise well to new data.

8.2 Hold-out test of the best model

The Logistic Regression model was trained on all 598 balanced samples in the training sets and used to predict the labels for the test set (150 rows: 75 fraudulent, 75 genuine). The results of this are shown in table 6 below.

The model detected 55 fraudulent transactions resulting in a high recall rate of 73.3%. However, out of the 87 transfers identified, only 55 of them were

fraudulent resulting in a lower precision score of 63.2%. Overall the accuracy score for the model was 65.3% and F1 score was 0.679.

Table 6: Hold-out test confusion matrix for the best model (Logistic Regression)

Actual \ Predicted	Not fraud	Fraud	Total
Not fraud	TN = 43	FP = 32	75
Fraud	FN = 20	TP = 55	75
Total	63	87	150

When trying to detect fraud, this is a more important factor to consider than reducing false positives. If there are any false negatives this will mean losing money at the bank. Whereas having some false positives will just cause some text checks and quick phone calls. The fact that we were able to achieve a high recall rate of 73.3% suggests that the model has learnt how to detect real fraud indicators, As suggested by the hold-out results shown in Table 6 and 7.

8.3 Comparison with the source paper

The following are the results of the models. The table in the paper shows that the two tree ensemble models have the highest accuracy. XGBoost had an F1-Score of 0.99 and Random Forest had a score of 0.97. Logistic Regression came in third place with a score of 0.93. However when looking at the test results Logistic Regression performed the best, this is because the tree ensemble models are less accurate due to having less data.

Table 7: Comparison with the source paper (10-fold validation)

Model	Paper accuracy	Our accuracy	Paper F1	Our F1
Logistic Regression	0.93	0.66	0.93	0.68
Decision Tree	0.95	0.57	0.95	0.55
Gaussian Naive Bayes	0.57	0.65	0.27	0.67
Random Forest	0.97	0.61	0.97	0.62
XGBoost	0.99	0.62	0.99	0.62
K-Nearest Neighbours	0.68	0.64	0.64	0.66

Table 7 compares our model results with those reported in the source paper using 10-fold validation. Our Logistic Regression achieves an F1-score of 0.68, while Gaussian Naive Bayes and KNN also show competitive results,

highlighting differences in performance due to preprocessing and class balancing.

- The results for the Gaussian Naive Bayes were quite different. It had an F1-Score of 0.27 in the paper but when we tested we had a much better score of 0.67. This was down to the feature scaling and smoothing variance.
- There were three main reasons that our results differed to the paper:
- Data size: The paper had reduced the data down to 2256 balanced data points in the training data. We only had 598 rows of data. Tree ensemble models require a lot of data to make sure they can find clean splits in the data without overfitting. The less data you have the more the trees will pick up on the noise in the data.
- Noise in the labels: The demo script adds fake noise into the labels by assigning fraud labels probabilistically based on the probability of behavioural actions. The Kaggle data has more deterministic patterns making it easier to split. Our data has more noise, making it harder for the models to learn.
- The lack of information on the hyperparameters meant we couldn't exactly replicate their results. They didn't say how much data they used for training/testing, what seed they used or what their full grid search was. When you fix the parameters and have less synthetic data you will get different scores. As we didn't have exact seeding we couldn't exactly match what they did, however our code allows you to try out new seeds quickly.

8.4 Overall discussion

- What we have learned from our experiments:
- There is an automated process which can produce, clean, under-sample, cross validate and save our data through simply using one command line.
- Simpler models can sometimes outperform complex models. Whilst boosting models performed well on the large benchmark leaderboard datasets, they did not perform as well on smaller noisy datasets.
- The code was scalable. If we simply put the full Kaggle dataset (online_payments.csv) in the data/raw/ folder, we can run our

pipeline on real data with no alteration to our code. All we need to do is drop the file into the correct folder and run our script.

9. Conclusion And Future Scope

A machine learning pipeline has been developed and implemented to detect potential fraud in online payments, following a similar approach to that taken by Al-dahasi et al [1]. In order to combat the issue of class imbalance in the local synthetic sample, we implemented Random Under-Sampling and pre-processing techniques to remove any identifying features and highly correlated balance fields. Using stratified 5-fold and 10-fold cross validation six different classifiers were tested. We have automated the process of pre-processing data, comparing models and testing the final model. This demonstrates how a full fraud detection pipeline can be constructed and tested using a local dataset.

The best performing model was Logistic Regression with L1 regularisation. With an F1 score of 0.6771 on 10-fold cross-validation. It was able to identify 73.3% of the fraudulent cases in the hold-out test set. This experiment demonstrates that a simple regularisation model can outperform more complex tree-based models when there is limited training data with noisy labelling. The command line tool and Streamlit application makes it easy to test the saved model. However, this should not be considered a representation of how the model would perform in real life on financial transactions.

The project can be extended in the following ways:

- Apply the pipeline to the full Kaggle online payments dataset and other external transaction datasets
- Compare the results using Random Under-Sampling techniques and over-sampling techniques. Additionally, compare the results using hybrid methods such as SMOTE based techniques and adjust the decision threshold to investigate the trade-off between precision and recall. [4] [20].

10. References

[1] E. M. Al-dahasi, R. K. Alsheikh, F. A. Khan, and G. Jeon, "Optimizing fraud detection in financial transactions with machine learning and imbalance mitigation," Expert Systems, vol. 42, no. 2, Art. no. e13682, 2025, doi: 10.1111/exsy.13682.

[2] Kaggle, "Online Payments Fraud Detection" dataset, 2019. [Online]. Available: https://www.kaggle.com/code/stbnlen/online-payments-fraud-detection/input

[3] N. S. Alfaiz and S. M. Fati, "Enhanced credit card fraud detection model using machine learning," Electronics, vol. 11, no. 4, Art. no. 662, 2022, doi: 10.3390/electronics11040662.

[4] S. Taneja, B. Suri, and C. Kothari, "Application of balancing techniques with ensemble approach for credit card fraud detection," in Proc. 2019 Int. Conf. on Computing, Power and Communication Technologies (GUCON), 2019, pp. 753–758.

[5] A. Gupta, M. C. Lohani, and M. Manchanda, "Financial fraud detection using Naive Bayes algorithm in highly imbalance data set," Journal of Discrete Mathematical Sciences and Cryptography, vol. 24, no. 5, pp. 1559–1572, 2021, doi: 10.1080/09720529.2021.1969733.

[6] S. Khosravi, M. Kargari, B. Teimourpour, A. Eshghi, and A. Aliabdi, "Using supervised machine learning approaches to detect fraud in the banking transaction network," in Proc. 2023 9th Int. Conf. on Web Research (ICWR), 2023, pp. 115–119, doi: 10.1109/ICWR57742.2023.10139083.

[7] Y. Alghofaili, A. Albattah, and M. A. Rassam, "A financial fraud detection model based on LSTM deep learning technique," Journal of Applied Security Research, vol. 15, pp. 498–516, 2020.

[8] W. Xiuguo and D. Shengyong, "An analysis on financial statement fraud detection for Chinese listed companies using deep learning," IEEE Access, vol. 10, pp. 22516–22532, 2022, doi: 10.1109/ACCESS.2022.3153478.

[9] T. Ashfaq, R. Khalid, A. S. Yahaya, S. Aslam, A. T. Azar, S. Alsafari, and I. A. Hameed, "A machine learning and blockchain based efficient fraud detection mechanism," Sensors, vol. 22, no. 19, Art. no. 7162, 2022, doi: 10.3390/s22197162.

[10] M. Zareapoor and P. Shamsolmoali, “Application of credit card fraud detection: Based on bagging ensemble classifier,” Procedia Computer Science, vol. 48, pp. 679–685, 2015, doi: 10.1016/j.procs.2015.04.201.

[11] M. Leo, S. Sharma, and K. Maddulety, “Machine learning in banking risk management: A literature review,” Risks, vol. 7, no. 1, Art. no. 29, 2019, doi: 10.3390/risks7010029.

[12] A. Mashrur, W. Luo, N. A. Zaidi, and A. Robles-Kelly, “Machine learning for financial risk management: A survey,” IEEE Access, vol. 8, pp. 203203–203223, 2020, doi: 10.1109/ACCESS.2020.3036322.

[13] D. Kalbande, P. Prabhu, A. Gharat, and T. Rajabally, “A fraud detection system using machine learning,” in Proc. 2021 12th Int. Conf. on Computing, Communication and Networking Technologies (ICCCNT), 2021, pp. 1–7, doi: 10.1109/ICCCNT51525.2021.9580102.

[14] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[15] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems, vol. 30, 2017.

[16] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

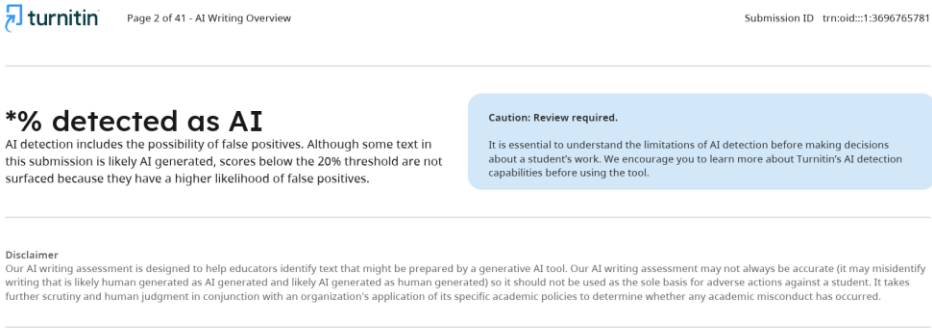
[17] G. Lemaître, F. Nogueira, and C. K. Aridas, “Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning,” Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.

[18] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[19] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, “PaySim: A financial mobile money simulator for fraud detection,” in Proc. 28th European Modeling and Simulation Symposium (EMSS), Larnaca, Cyprus, 2016, pp. 249–255. [Online]. Available: https://www.msc-les.org/proceedings/emss/2016/EMSS2016_249.pdf

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002, doi: 10.1613/jair.953.

11.Plag Report



turnitin Page 2 of 41 - AI Writing Overview Submission ID trn:oid:::1:3696765781

*% detected as AI

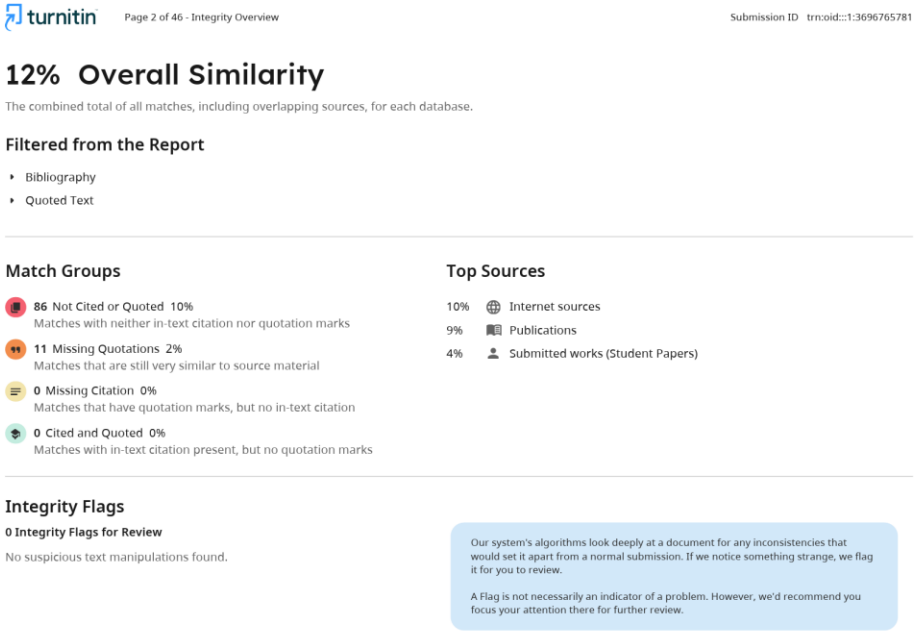
AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



turnitin Page 2 of 46 - Integrity Overview Submission ID trn:oid:::1:3696765781

12% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

- **86** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **11** Missing Quotations 2%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
9% Publications
4% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.